

PHP MIDDLE

Вопросы и ответы для собеседования

100 вопросов по PHP 8.x, ООП, архитектуре, HTTP, базам данных, Redis, очередям, фреймворкам, тестированию и производительности

*Формат ответов рассчитан на устное техническое интервью:
кратко, по существу, с практическими примерами и ключевыми терминами.*

Уровень: Middle PHP Developer

Содержание

1. PHP 8.x и основы языка	10 вопросов
2. ООП, SOLID и дизайн	10 вопросов
3. Composer, PSR и контейнер зависимостей	10 вопросов
4. HTTP, REST и безопасность	10 вопросов
5. MySQL, PostgreSQL и транзакции	10 вопросов
6. Кэширование, Redis и очереди	10 вопросов
7. Laravel, Symfony и Yii2	10 вопросов
8. Тестирование и качество кода	10 вопросов
9. Архитектура, микросервисы и highload	10 вопросов
10. Производительность, Docker и практические вопросы	10 вопросов

Как пользоваться документом: сначала попробуйте ответить самостоятельно за 1–2 минуты, затем сравните со структурой ответа. Примеры кода предназначены для обсуждения подхода, а не для механического заучивания.

1. PHP 8.x и основы языка

1. Какие особенности современного PHP важны для backend-разработчика?

Ответ: Современный PHP — строго развивающийся серверный язык с JIT, типизированными свойствами, union/intersection types, attributes, enums, readonly-сущностями и улучшенной системой типов. Для Middle-разработчика важнее не перечислить синтаксис, а показать умение применять типизацию, Composer, PSR, тестирование и профилирование в реальном проекте.

PHP остаётся интерпретируемым языком с моделью «один запрос — один процесс/воркер» в классическом PHP-FPM. Состояние между запросами обычно хранится во внешних системах: БД, Redis, файловом или объектном хранилище.

2. Чем строгая типизация отличается от обычного режима?

Ответ: Директива `declare(strict_types=1)` влияет на скалярные аргументы и возвращаемые значения при вызове функции из текущего файла. В строгом режиме PHP не выполняет часть неявных приведений, поэтому ошибка обнаруживается раньше.

Это не превращает PHP в полностью статически типизированный язык: типы по-прежнему проверяются во время выполнения. Для более ранней диагностики используют PHPStan или Psalm.

```
1  <?php
2  declare(strict_types=1);
3
4  function total(int $price, int $count): int
5  {
6      return $price * $count;
7  }
8
9  echo total(100, 3);
```

3. Чем == отличается от ===?

Ответ: Оператор `==` сравнивает значения с приведением типов, а `===` требует совпадения и значения, и типа. В прикладном коде безопаснее по умолчанию использовать строгое сравнение, особенно для идентификаторов, статусов, булевых значений и результатов поиска.

Классическая ловушка: `strpos()` может вернуть 0, поэтому проверка `if (!$position)` ошибочна. Нужна проверка `$position === false`.

```
1  <?php
2  $position = strpos('php middle', 'php');
3
4  if ($position === false) {
5      echo 'Не найдено';
6  }
```

4. Что такое null coalescing и nullsafe operator?

Ответ: Оператор `??` возвращает правую часть, если левая переменная не определена или равна `null`. Оператор `?->` позволяет безопасно продолжить цепочку вызовов, если объект может быть `null`.

Они сокращают проверки, но не должны скрывать ошибочное состояние модели. Если объект обязан существовать по бизнес-правилам, лучше выбросить исключение, а не молча получить `null`.

5. Чем include, require и их варианты _once отличаются?

Ответ: `include` выдаёт предупреждение и продолжает выполнение, `require` при неудаче завершает выполнение ошибкой. Варианты `include_once` и `require_once` дополнительно предотвращают повторное подключение файла.

В современных приложениях прямые подключения почти всегда заменяет автозагрузка Composer по PSR-4. `require` обычно остаётся только для bootstrap-файла `vendor/autoload.php`.

6. Как работают пространства имён?

Ответ: Namespace предотвращает конфликты имён классов, функций и констант. Импорт через `use` влияет только на читаемость имени в текущем файле и не загружает класс сам по себе.

Структура namespace обычно соответствует каталогу по PSR-4: например `App\Domain\Order` находится в `src/Domain/Order.php`.

7. Что такое генераторы и `yield`?

Ответ: Генератор возвращает значения по одному и не хранит весь набор в памяти. Функция с `yield` возвращает объект `Generator`, который можно перебирать через `foreach`.

Это полезно для потоковой обработки больших файлов, выгрузок и выборок. Однако генератор не делает запрос к БД автоматически эффективным: важно, чтобы драйвер и ORM также не буферизовали весь результат.

```
1  <?php
2  function readLines(string $file): Generator
3  {
4      $handle = fopen($file, 'rb');
5      try {
6          while (($line = fgets($handle)) !== false) {
7              yield rtrim($line);
8          }
9      } finally {
10         fclose($handle);
11     }
12 }
```

8. Что такое `match` и чем он отличается от `switch`?

Ответ: `match` является выражением, возвращает значение и использует строгое сравнение. Он не требует `break` и не допускает случайного проваливания между ветками.

`match` удобно применять для преобразования enum или статуса в действие. Для сложной бизнес-логики лучше использовать полиморфизм или отдельную стратегию.

```
1  <?php
2  $statusCode = match ($status) {
3      OrderStatus::New => 201,
4      OrderStatus::Paid => 200,
5      OrderStatus::Cancelled => 409,
6  };
```

9. Что такое `named arguments` и когда они опасны?

Ответ: Именованные аргументы позволяют передавать параметры по имени и пропускать необязательные параметры. Они делают вызов понятнее, но превращают имена параметров публичного метода в часть внешнего контракта.

Переименование параметра может сломать потребителей. Поэтому в библиотечном коде имена параметров следует менять осторожно.

10. Что такое `attributes`?

Ответ: `Attributes` — структурированные метаданные, доступные через `Reflection`. Фреймворки используют их для маршрутов, ORM-маппинга, сериализации, валидации и DI.

Атрибут сам по себе ничего не выполняет: нужна инфраструктура, которая его прочитает и применит. Не стоит переносить в атрибуты сложную бизнес-логику.

2. ООП, SOLID и дизайн

11. Чем инкапсуляция отличается от сокрытия данных?

Ответ: Сокрытие данных ограничивает прямой доступ к внутреннему состоянию. Инкапсуляция шире: объект объединяет состояние и поведение и защищает инварианты через публичные методы.

Хорошая сущность не просто содержит public-поля, а гарантирует допустимое состояние: например `Order::pay()` проверяет, что заказ ещё не оплачен.

12. Когда использовать наследование, а когда композицию?

Ответ: Наследование оправдано при устойчивом отношении «является» и общем поведенческом контракте. Композиция связывает объекты через зависимости и обычно лучше переносит изменения.

Практическое правило: расширять поведение сервисов через стратегии, декораторы и делегирование, а наследование оставлять для действительно общей абстракции.

13. Что такое полиморфизм?

Ответ: Полиморфизм позволяет работать с разными реализациями через общий интерфейс. Код зависит от контракта, а конкретная реализация выбирается при сборке приложения.

Например `PaymentProcessor` принимает `PaymentGateway`, поэтому `StripeGateway` можно заменить тестовым `FakeGateway` без изменения бизнес-логики.

```
1  <?php
2  interface PaymentGateway
3  {
4      public function charge(Money $amount): PaymentId;
5  }
6
7  final class CheckoutService
8  {
9      public function __construct(
10         private PaymentGateway $gateway,
11     ) {}
12 }
```

14. Объясните принцип единственной ответственности.

Ответ: SRP означает, что у модуля должна быть одна причина для изменения. Это не правило «один метод на класс», а разделение разных политик: бизнес-расчёта, хранения, доставки и форматирования.

Если класс одновременно валидирует заказ, пишет SQL, отправляет письмо и строит JSON, его сложно тестировать и изменять.

15. Что означает Open/Closed Principle?

Ответ: Компонент должен позволять расширять поведение без постоянного изменения стабильного кода. Обычно это достигается через интерфейсы, композицию и регистрацию обработчиков.

Но не стоит заранее создавать абстракцию для каждой потенциальной вариации. Сначала выявляют реальную точку изменения.

16. Что означает Liskov Substitution Principle?

Ответ: Объект подтипа должен быть взаимозаменяем с базовым типом без нарушения ожидаемого поведения. Нельзя усиливать предусловия, ослаблять постусловия или неожиданно менять семантику метода.

Если наследник `Repository::save()` внезапно только логирует объект и не сохраняет его, контракт нарушен.

17. Что такое Interface Segregation Principle?

Ответ: Клиент не должен зависеть от методов, которые ему не нужны. Несколько небольших интерфейсов лучше одного универсального, если потребители используют разные части поведения.

Например `Reader` и `Writer` удобнее большого `Storage` с десятком операций.

18. Что означает Dependency Inversion Principle?

Ответ: Высокоуровневая бизнес-логика зависит от абстракций, а инфраструктура реализует эти абстракции. Интерфейс часто располагается рядом с потребителем, а не рядом с конкретной БД или API. Это уменьшает связность и упрощает тестирование, но не требует создавать интерфейс для каждого класса.

19. Чем DTO отличается от Entity и Value Object?

Ответ: DTO переносит данные между слоями и обычно не содержит бизнес-поведения. Entity имеет идентичность и жизненный цикл. Value Object определяется значением, обычно неизменяем и проверяет собственные инварианты.

Email, Money и DateRange часто являются Value Object; Order — Entity; CreateOrderData — DTO.

```
1  <?php
2  final readonly class Money
3  {
4      public function __construct(
5          public int $amount,
6          public string $currency,
7      ) {
8          if ($amount < 0) {
9              throw new InvalidArgumentException('Negative amount');
10         }
11     }
12 }
```

20. Что такое immutable и readonly объекты?

Ответ: Неизменяемый объект не меняет наблюдаемое состояние после создания. readonly в PHP запрещает повторную запись свойства после инициализации, но не гарантирует глубокую неизменяемость вложенных объектов.

Immutable-модель уменьшает число скрытых побочных эффектов и особенно удобна для Value Object.

3. Composer, PSR и контейнер зависимостей

21. Как работает Composer?

Ответ: Composer разрешает зависимости по composer.json, фиксирует конкретные версии в composer.lock и создаёт автозагрузчик vendor/autoload.php. Для приложения lock-файл обычно коммитят, чтобы окружения ставили одинаковые версии.

В production используют composer install, а не update; часто добавляют --no-dev и --classmap-authoritative после проверки совместимости.

22. Как работает PSR-4 autoloading?

Ответ: PSR-4 сопоставляет префикс namespace с каталогом. При обращении к классу автозагрузчик преобразует оставшуюся часть имени в путь к файлу.

Например App\Service\Mailer при mapping App\ => src/ ищется в src/Service/Mailer.php.

```
1  {
2      "autoload": {
3          "psr-4": {
4              "App\\": "src/"
5          }
6      }
7  }
```

23. Что такое semantic versioning?

Ответ: Версия MAJOR.MINOR.PATCH означает несовместимые изменения, обратно совместимые возможности и исправления. Ограничение ^2.3 обычно допускает версии до 3.0.0, а ~2.3 — обновления в рамках более узкого диапазона.

Важно оценивать не только constraint, но и changelog, поддержку PHP и транзитивные зависимости.

24. Что такое PSR-3?

Ответ: PSR-3 задаёт общий интерфейс логгера с уровнями emergency, alert, critical, error, warning, notice, info и debug. Контекст передаётся массивом и может содержать placeholder-значения.

Бизнес-код зависит от LoggerInterface, а Monolog или другой адаптер подключается в инфраструктуре.

25. Что задают PSR-7, PSR-15, PSR-17 и PSR-18?

Ответ: PSR-7 описывает immutable HTTP request/response и streams. PSR-15 — server request handlers и middleware. PSR-17 — фабрики HTTP-объектов. PSR-18 — интерфейс HTTP-клиента.

Совместимость по этим стандартам позволяет заменять реализации и переиспользовать middleware.

26. Что такое Dependency Injection Container?

Ответ: Контейнер создаёт объекты и связывает их зависимости. Хороший код не обращается к контейнеру из бизнес-логики; зависимости передаются через конструктор.

Service Locator, когда объект сам запрашивает container->get(), скрывает зависимости и усложняет тесты.

27. Чем autowiring отличается от явной конфигурации?

Ответ: Autowiring определяет зависимости по типу hints и сокращает конфигурацию. Явная конфигурация нужна, когда интерфейс имеет несколько реализаций, требуются scalar-параметры, фабрика или декоратор.

На собеседовании важно отметить, что магия контейнера не заменяет ясную архитектуру.

```
1  <?php
2  final class OrderService
3  {
4      public function __construct(
5          private OrderRepository $orders,
6          private LoggerInterface $logger,
7      ) {}
8  }
```

28. Что такое service provider или bundle/extension?

Ответ: Это механизм регистрации сервисов и конфигурации модуля в контейнере. Он связывает интерфейсы с реализациями, задаёт factories, decorators и параметры окружения.

Инфраструктурная регистрация должна быть отделена от доменной логики.

29. Почему нельзя хранить секреты в composer.json или репозитории?

Ответ: Секреты должны приходить из переменных окружения или secret manager. История Git сохраняет удалённые значения, а расpackage-файлы часто распространяются.

При утечке секрет необходимо отозвать и заменить, а не только удалить из текущей версии.

30. Как проверить качество зависимостей?

Ответ: Оценивают поддержку, частоту релизов, лицензию, совместимость, открытые уязвимости и размер транзитивного дерева. Используют composer audit и обновляют зависимости контролируемо.

Небольшую функцию иногда дешевле реализовать локально, чем добавлять тяжёлый пакет.

4. HTTP, REST и безопасность

31. Из чего состоит HTTP-запрос?

Ответ: HTTP-запрос содержит метод, URI, версию протокола, заголовки и необязательное тело. Сервер отвечает статусом, заголовками и телом.

HTTP сам по себе stateless, а сессии реализуются через cookie или токен и внешнее хранилище.

32. Какие HTTP-методы идемпотентны?

Ответ: GET, PUT, DELETE, HEAD и OPTIONS определены как идемпотентные по семантике: повтор одинакового запроса должен иметь тот же эффект на состояние. POST обычно не идемпотентен.

Для платежей и создания ресурсов идемпотентность часто добавляют через Idempotency-Key и таблицу обработанных запросов.

33. Чем PUT отличается от PATCH?

Ответ: PUT обычно заменяет полное представление ресурса, PATCH применяет частичное изменение. На практике контракт должен быть явно задокументирован, включая поведение отсутствующих и null-полей.

PATCH может использовать JSON Merge Patch или JSON Patch, но формат необходимо согласовать.

34. Какие коды ответа важно знать?

Ответ: 200 — успех, 201 — ресурс создан, 204 — успех без тела, 400 — некорректный запрос, 401 — не аутентифицирован, 403 — доступ запрещён, 404 — не найдено, 409 — конфликт, 422 — ошибка валидации, 429 — лимит, 500 — внутренняя ошибка.

Код должен отражать семантику, а тело ошибки — иметь стабильный машиночитаемый формат.

35. Что такое REST?

Ответ: REST — архитектурный стиль с ресурсами, единообразным интерфейсом, stateless-взаимодействием и возможностью кэширования. REST не равен «JSON поверх HTTP», хотя часто реализуется именно так.

URI обычно описывает существительное, а действие выражается HTTP-методом.

36. Как устроен JWT?

Ответ: JWT состоит из header, payload и signature. Подпись подтверждает целостность, но payload по умолчанию не шифруется. Проверяют алгоритм, подпись, exp, nbf, iss, aud и уникальность токена при необходимости.

Короткоживущий access token часто дополняют refresh token, который можно отозвать.

37. Чем OAuth 2.0 отличается от OpenID Connect?

Ответ: OAuth 2.0 делегирует доступ к ресурсам, а OpenID Connect добавляет слой аутентификации и ID Token. Authorization Code с PKCE — предпочтительный поток для публичных клиентов.

Нельзя использовать access token как доказательство личности без понимания issuer и audience.

38. Как защищаться от SQL Injection?

Ответ: Использовать подготовленные запросы и bind parameters. Нельзя склеивать пользовательские значения в SQL. Имена колонок и направление сортировки параметризовать нельзя, их проверяют по allowlist.

ORM уменьшает риск, но raw expressions и неверная динамическая сборка остаются опасными.

```
1  <?php
2  $stmt = $pdo->prepare(
3      'SELECT * FROM users WHERE email = :email'
4  );
5  $stmt->execute(['email' => $email]);
```

39. Как защищаться от XSS и CSRF?

Ответ: От XSS защищают контекстное экранирование вывода, безопасные шаблонизаторы, CSP и запрет опасного HTML. CSRF предотвращают SameSite cookie, CSRF token и проверка Origin/Referer для state-changing запросов.

JWT в Authorization header снижает классический CSRF-риск, но XSS всё равно может похитить токен.

40. Как хранить пароли?

Ответ: Пароли хэшируют password_hash() с современным алгоритмом и проверяют password_verify(). Соль создаётся автоматически. Шифровать пароль обратимым способом нельзя.

После успешного входа полезно проверять `password_needs_rehash()` и обновлять параметры хэша.

```
1 <?php
2 $hash = password_hash($password, PASSWORD_DEFAULT);
3
4 if (!password_verify($input, $hash)) {
5     throw new AuthenticationException();
6 }
```

5. MySQL, PostgreSQL и транзакции

41. Что такое ACID?

Ответ: Atomicity гарантирует выполнение транзакции целиком или откат; Consistency — соблюдение ограничений; Isolation — контролируемое взаимодействие параллельных транзакций; Durability — сохранность подтверждённых изменений.

Реальная гарантия зависит от СУБД, уровня изоляции и настроек хранения.

42. Какие уровни изоляции существуют?

Ответ: Read Uncommitted, Read Committed, Repeatable Read и Serializable различаются допустимыми аномалиями. Чем сильнее изоляция, тем выше потенциальная стоимость блокировок и конфликтов.

Нужно знать поведение конкретной СУБД: PostgreSQL и MySQL реализуют MVCC по-разному.

43. Что такое MVCC?

Ответ: MVCC хранит версии строк, позволяя читателям не блокировать писателей в типичных сценариях. Видимость версии определяется snapshot транзакции.

Старые версии должны очищаться: в PostgreSQL это связано с VACUUM, а долгие транзакции могут мешать очистке.

44. Чем INNER JOIN отличается от LEFT JOIN?

Ответ: INNER JOIN возвращает только совпавшие строки. LEFT JOIN сохраняет все строки левой таблицы и заполняет правую часть null при отсутствии совпадения.

Условие по правой таблице в WHERE может случайно превратить LEFT JOIN в эквивалент INNER JOIN; иногда условие нужно перенести в ON.

45. Что такое составной индекс и правило левого префикса?

Ответ: Составной индекс хранит несколько колонок в заданном порядке. Он эффективен для запросов, использующих ведущие колонки индекса.

Порядок выбирают по фильтрации, диапазонам, сортировке и селективности, проверяя план через EXPLAIN ANALYZE.

46. Почему индекс может не использоваться?

Ответ: Причины: низкая селективность, функция или приведение над колонкой, несовпадение порядка составного индекса, слишком большой диапазон, устаревшая статистика или более дешёвый последовательный проход.

Нельзя заставлять индекс без измерений: оптимизатор может быть прав.

47. Что такое N+1?

Ответ: После одного запроса списка выполняется отдельный запрос для каждого элемента. Проблема резко увеличивает latency и нагрузку на БД.

Решения: JOIN, eager loading, пакетная загрузка по IN, DataLoader или изменение границы агрегата.

48. Как правильно выполнять транзакцию в PHP?

Ответ: Начать транзакцию, выполнить операции, commit при успехе и rollback при исключении. Внешние HTTP-вызовы не стоит держать внутри долгой DB-транзакции.

При deadlock или serialization failure допустим ограниченный retry с backoff, если операция идиempотентна.

```
1  <?php
2  $pdo->beginTransaction();
3  try {
4      $orders->save($order);
5      $outbox->append($event);
6      $pdo->commit();
7  } catch (Throwable $e) {
8      $pdo->rollBack();
9      throw $e;
10 }
```

49. Что такое optimistic и pessimistic locking?

Ответ: Pessimistic locking заранее блокирует строку, например SELECT ... FOR UPDATE. Optimistic locking проверяет version или updated_at при UPDATE.

Оптимистический подход подходит при редких конфликтах; пессимистический — когда конфликт вероятен и операция короткая.

50. Как делать безопасные миграции на большой таблице?

Ответ: Изменения выполняют поэтапно: добавить nullable-колодку, развернуть код, заполнить данные батчами, добавить ограничение и удалить старую колонку позже.

Нужно учитывать блокировки DDL, размер индекса, репликацию и обратную совместимость во время rolling deploy.

6. Кэширование, Redis и очереди

51. Какие стратегии кэширования вы знаете?

Ответ: Cache-aside: приложение сначала читает кэш, при промахе — источник и записывает результат. Write-through обновляет кэш вместе с хранилищем, write-behind откладывает запись.

Для большинства web-сценариев применяют cache-aside с TTL и явной инвалидцией.

52. Что такое cache stampede?

Ответ: После истечения популярного ключа множество запросов одновременно идёт в БД. Защита: mutex/lock, probabilistic early expiration, stale-while-revalidate и небольшой jitter к TTL.

Лок должен иметь таймаут и безопасное освобождение.

53. Когда использовать Redis?

Ответ: Redis подходит для кэша, сессий, rate limit, распределённых locks, counters, streams и короткоживущего состояния. Это не автоматическая замена реляционной БД.

Нужно понимать persistence, eviction policy, memory limits и поведение при failover.

54. Чем Redis String, Hash, Set, Sorted Set и Stream отличаются?

Ответ: String хранит значение или счётчик; Hash — набор полей объекта; Set — уникальные элементы; Sorted Set — элементы с score; Stream — журнал сообщений с consumer groups.

Выбор структуры влияет на атомарность, память и сложность операций.

55. Как реализовать rate limiting?

Ответ: Популярные алгоритмы: fixed window, sliding window, token bucket и leaky bucket. В Redis операции выполняют атомарно через INCR/EXPIRE или Lua script.

Ключ обычно включает клиента и endpoint, а ответ содержит 429 и информацию о повторе.

```
1 <?php
2 $key = 'rate:' . $clientId;
3 $count = $redis->incr($key);
4 if ($count === 1) {
5     $redis->expire($key, 60);
6 }
7 if ($count > 100) {
8     throw new TooManyRequestsException();
9 }
```

56. Что такое очередь сообщений?

Ответ: Очередь отделяет producer от consumer, сглаживает пики и позволяет обрабатывать задачи асинхронно. Нужно определить ack, retry, dead-letter queue, порядок и идиempотентность.

Публикация сообщения ещё не означает успешную обработку.

57. Чем at-most-once, at-least-once и exactly-once отличаются?

Ответ: At-most-once допускает потерю, но не повтор. At-least-once допускает повтор и требует идиempотентного consumer. Exactly-once как сквозная гарантия сложна и обычно достигается комбинацией транзакций и дедупликации.

На практике чаще проектируют at-least-once + idempotency.

58. Как реализовать повторную обработку?

Ответ: Retry делают ограниченным, с exponential backoff и jitter. Постоянные ошибки отправляют в DLQ, а временные повторяют.

Важно различать типы ошибок и не создавать бесконечный цикл, который перегружает зависимость.

59. Что такое transactional outbox?

Ответ: Бизнес-изменение и запись события в outbox-таблицу сохраняются одной транзакцией. Worker читает outbox и публикует событие, после чего отмечает его доставленным.

Это устраняет окно между commit в БД и публикацией, но consumer всё равно должен быть идиempотентным.

```
1 <?php
2 // В одной транзакции: бизнес-данные + outbox
3 $orderRepository->save($order);
4 $outboxRepository->add(new OrderCreated(
5     id: Uuid::v7(),
6     orderId: $order->id(),
7 ));
```

60. Как гарантировать идиempотентность consumer?

Ответ: Сообщение имеет уникальный id. Consumer атомарно проверяет или записывает этот id вместе с бизнес-изменением. Повторное сообщение обнаруживается и не применяется эффект второй раз.

Таблица дедупликации должна очищаться с учётом срока повторной доставки.

7. Laravel, Symfony и Yii2

61. Как устроен жизненный цикл запроса в Laravel?

Ответ: HTTP-запрос входит через public/index.php, загружается Composer, создаётся Application, выполняются bootstrapper-ы, middleware, router и controller. Ответ проходит обратную цепочку middleware и отправляется клиенту.

Service providers регистрируют и загружают сервисы контейнера.

62. Что такое Eloquent и его ограничения?

Ответ: Eloquent — Active Record ORM: модель объединяет данные и операции хранения. Он ускоряет CRUD, relations, scopes и events.

На сложных отчётах и высоконагруженных запросах важно контролировать SQL, N+1, memory hydration и иногда использовать Query Builder или raw SQL.

63. Чем service container Laravel полезен?

Ответ: Он разрешает зависимости, поддерживает bindings, singleton, contextual binding, factories и auto-wiring. Контроллеры и команды получают зависимости через конструктор.

Нельзя превращать Facade или app() в скрытый глобальный Service Locator в доменной логике.

64. Что такое middleware в Laravel/Symfony?

Ответ: Middleware или kernel listener обрабатывает запрос до и после приложения: auth, rate limit, locale, correlation id, logging.

Бизнес-правила не следует помещать в middleware, потому что они становятся неявными и труднее тестируются.

65. Как работают Laravel queues?

Ответ: Job сериализуется и помещается в backend очереди, worker извлекает и вызывает handle(). Настраиваются tries, timeout, backoff, failed jobs и уникальность.

В Job нельзя бездумно сериализовать большие ORM-графы; лучше передавать идентификатор и перечитать актуальные данные.

```
1  <?php
2  final class SendInvoice implements ShouldQueue
3  {
4      public int $tries = 5;
5
6      public function __construct(public int $orderId) {}
7
8      public function handle(InvoiceService $service): void
9      {
10         $service->send($this->orderId);
11     }
12 }
```

66. Что такое Symfony HttpKernel и EventDispatcher?

Ответ: HttpKernel преобразует Request в Response и публикует события на этапах обработки. Слушатели могут добавить security, routing, controller resolution и exception handling.

Приоритеты слушателей важны, но чрезмерная событийность ухудшает прозрачность потока.

67. Чем Symfony service configuration отличается от Laravel?

Ответ: Оба используют DI container и autowiring. Symfony активно компилирует контейнер и предоставляет tags, compiler passes и aliases; Laravel чаще конфигурируется через service providers и bindings.

Архитектурные принципы одинаковы: зависимости явны, infrastructure отделена.

68. Как устроен Yii2 DI container?

Ответ: Yii::\$container хранит definitions и singletons, а объект создаётся с разрешением constructor dependencies. Компоненты приложения доступны через service locator Yii::\$app.

В новом коде лучше передавать сервисы явно, а не обращаться к Yii::\$app повсюду.

69. Что такое ActiveRecord и Repository pattern?

Ответ: ActiveRecord удобен для CRUD и простых доменов, но связывает модель с БД. Repository скрывает детали хранения за контрактом и полезен, когда доменная модель должна быть независима.

Repository не обязан механически дублировать каждый метод ORM.

70. Где размещать валидацию?

Ответ: Формат и обязательность входа проверяются в Request/Form DTO. Бизнес-инварианты проверяет доменная модель или application service. Ограничения целостности также закрепляют в БД.

Одна проверка на HTTP-уровне недостаточна, если операция может вызываться из CLI или очереди.

8. Тестирование и качество кода

71. Чем unit test отличается от integration test?

Ответ: Unit-тест проверяет небольшой компонент в изоляции и выполняется быстро. Integration-тест проверяет взаимодействие с БД, файловой системой, контейнером или внешним протоколом.

Нужны оба типа: unit хорошо локализует логику, integration выявляет ошибки конфигурации и контракта.

72. Что такое Arrange-Act-Assert?

Ответ: Arrange подготавливает данные и зависимости, Act выполняет одно действие, Assert проверяет результат и побочные эффекты. Такая структура делает тест читаемым.

Один тест должен описывать одну значимую ситуацию, а не десяток несвязанных сценариев.

73. Что такое mock, stub, fake и spy?

Ответ: Stub возвращает заданные данные; mock проверяет ожидаемые взаимодействия; fake содержит упрощённую рабочую реализацию; spy записывает вызовы.

Избыточные mocks делают тест привязанным к реализации. Чаще полезнее fake repository или проверка результата.

74. Что тестировать в контроллере?

Ответ: Маршрут, auth, валидацию, преобразование запроса, статус и JSON-контракт. Основную бизнес-логику лучше тестировать в application/domain сервисе.

Feature-тест не должен повторять все unit-сценарии, но должен проверить ключевую интеграцию.

75. Что такое data provider?

Ответ: Data provider передаёт наборы аргументов одному тесту. Он полезен для граничных значений, форматов и таблиц решений.

Название dataset должно описывать случай, чтобы падение было понятно без отладки.

```
1  <?php
2  final class FrozenClock implements Clock
3  {
4      public function __construct(private DateTimeImmutable $now) {}
5
6      public function now(): DateTimeImmutable
7      {
8          return $this->now;
9      }
10 }
```

76. Как тестировать код с текущим временем?

Ответ: Время передают через Clock interface или используют контролируемый test clock. Нельзя разбросать new DateTimeImmutable() по доменной логике.

Это делает тесты детерминированными и позволяет проверять expiry и переходы дат.

77. Что такое mutation testing?

Ответ: Mutation testing изменяет код небольшими мутациями и проверяет, обнаружат ли тесты ошибку. Если мутант выжил, assertions могут быть слабыми.

Инструмент полезен для критической логики, но дороже обычного покрытия.

78. Что показывает code coverage?

Ответ: Coverage показывает, какие строки или ветки были выполнены, но не доказывает качество assertions. 100% coverage может ничего не гарантировать.

Важнее покрыть риски, бизнес-правила и границы, а процент использовать как вспомогательную метрику.

79. Как использовать PHPStan/Psalm?

Ответ: Статический анализ проверяет типы, недостижимый код, неправильные вызовы и generics-аннотации без запуска приложения. Уровень повышают постепенно и фиксируют baseline только для legacy.

Новые ошибки не должны добавляться в baseline.

```
1  <?php
2  /** @return list<Order> */
3  public function findActive(UserId $userId): array
4  {
5      return $this->orders->findBy([
6          'userId' => $userId,
7          'active' => true,
8      ]);
9  }
```

80. Что проверять на code review?

Ответ: Корректность бизнес-логики, границы ответственности, безопасность, SQL, транзакции, идемпотентность, обработку ошибок, тесты, читаемость и наблюдаемость.

Review должен обсуждать риски и контракт, а не только стиль, который лучше автоматизировать formatter-ом.

9. Архитектура, микросервисы и highload

81. Что такое слоистая архитектура?

Ответ: Обычно выделяют presentation, application, domain и infrastructure. Зависимости направлены к бизнес-логике, а детали БД и HTTP находятся снаружи.

Слои нужны не ради каталогов, а чтобы изменения инфраструктуры не разрушали правила предметной области.

82. Что такое Clean Architecture?

Ответ: Clean Architecture ставит use cases и entities в центр. Внешние адаптеры реализуют порты, определённые внутренними слоями.

На небольшом CRUD-проекте чрезмерное количество абстракций может быть лишним; архитектура должна соответствовать сложности.

83. Монолит или микросервисы?

Ответ: Монолит проще разворачивать, тестировать и поддерживать транзакции. Микросервисы дают независимое масштабирование и автономию команд, но добавляют сеть, наблюдаемость, eventual consistency и сложность эксплуатации.

Часто разумно начать с модульного монолита и выделять сервисы по реальной границе.

84. Как выбрать границы сервиса?

Ответ: Ориентируются на bounded context, владение данными, независимый темп изменений и командную ответственность. Нельзя делить только по техническим слоям вроде «сервис пользователей» без анализа домена.

Сервис должен владеть своей моделью и избегать общей БД как постоянного контракта.

85. Что такое eventual consistency?

Ответ: Изменения между компонентами распространяются не мгновенно, поэтому некоторое время данные могут расходиться. Система должна определять допустимое окно, повторные попытки и способы компенсации.

Пользовательский интерфейс должен корректно показывать промежуточный статус.

86. Что такое Saga?

Ответ: Saga координирует последовательность локальных транзакций с компенсирующими действиями. Choreography использует события, orchestration — центрального координатора.

Компенсация не всегда является точным rollback: например возврат платежа — отдельная бизнес-операция.

87. Что такое circuit breaker, timeout и bulkhead?

Ответ: Timeout ограничивает ожидание, circuit breaker временно прекращает вызовы проблемной зависимости, bulkhead изолирует ресурсы между группами запросов. Вместе они предотвращают каскадный отказ.

Retry без timeout и лимита может только усилить аварию.

```
1  <?php
2  final class RetryPolicy
3  {
4      public function delay(int $attempt): int
5      {
6          return min(30, 2 ** $attempt) + random_int(0, 3);
7      }
8  }
```

88. Как масштабировать PHP-приложение?

Ответ: Приложение делают stateless, добавляют PHP-FPM/worker instances за load balancer, выносят сессии и кэш, используют read replicas и очереди.

Сначала измеряют bottleneck: CPU, БД, I/O, внешние API или lock contention.

89. Как проектировать API для обратной совместимости?

Ответ: Изменения делают additive: добавляют необязательные поля, не меняют смысл существующих, не удаляют enum-значения без версии. Consumer-driven contract tests помогают обнаружить поломки.

Версионирование в URI или header не отменяет дисциплину эволюции.

90. Как обрабатывать большой импорт?

Ответ: Файл читают потоково, строки валидируют и обрабатывают батчами, тяжёлую работу отправляют в очередь, прогресс сохраняют. Ошибки собирают отдельно, а операция должна быть возобновляемой.

Нельзя загружать весь XLSX/CSV и все сущности ORM в память одновременно.

10. Производительность, Docker и практические вопросы

91. Как искать узкое место производительности?

Ответ: Сначала определить симптом и метрику, затем собрать traces, slow queries, CPU/heap profile и тайминги внешних вызовов. Оптимизация без измерений часто переносит проблему.

Сравнивают p50, p95, p99, throughput и error rate до и после изменения.

92. Что такое OPcache?

Ответ: OPcache хранит скомпилированный bytecode PHP в shared memory, чтобы не разбирать файлы на каждый запрос. В production настраивают память, количество файлов и стратегию проверки timestamp.

После immutable deploy можно отключить частую проверку изменений и перезапускать workers при релизе.

93. Когда JIT помогает PHP?

Ответ: JIT может ускорить CPU-intensive вычисления, но типичное web-приложение чаще ограничено БД, сетью и I/O. Поэтому эффект обычно меньше, чем от индексов, кэша или устранения N+1.

Решение принимают по benchmark конкретной нагрузки.

94. Как уменьшить память при обработке ORM-данных?

Ответ: Выбирать только нужные поля, использовать cursor/iterate, очищать Unit of Work батчами, избегать eager loading больших графов и не накапливать результаты.

Важно учитывать, что некоторые драйверы буферизуют результат независимо от iterator API.

```
1 FROM composer:2 AS vendor
2 WORKDIR /app
3 COPY composer.* ./
4 RUN composer install --no-dev --no-interaction --prefer-dist
5
6 FROM php:8.3-fpm-alpine
7 WORKDIR /var/www/html
8 COPY --from=vendor /app/vendor ./vendor
9 COPY . .
10 USER www-data
```

95. Как устроен PHP-FPM?

Ответ: Master управляет pool worker-процессов, каждый worker обрабатывает запросы. Режимы pm static, dynamic и ondemand определяют число процессов.

Настройки рассчитывают по памяти одного worker, latency и допустимому параллелизму, иначе сервер уходит в swap.

96. Что должно быть в Dockerfile PHP-приложения?

Ответ: Фиксированный base image, системные зависимости, PHP extensions, Composer install, непривилегированный пользователь и минимальный runtime image. Multi-stage build отделяет сборку от запуска.

Секреты нельзя запекать в image, а контейнер должен писать логи в stdout/stderr.

97. Чем Docker image отличается от container?

Ответ: Image — неизменяемый шаблон слоёв, container — запущенный экземпляр image с writable layer. Данные, которые должны пережить пересоздание, хранят в volume или внешнем сервисе.

Изменение внутри контейнера не является способом доставки релиза.

98. Как безопасно выполнить zero-downtime deploy?

Ответ: Запустить новую версию, пройти health checks, постепенно переключить traffic, затем остановить старую. Миграции должны быть обратно совместимыми, workers — корректно завершать текущие задачи. Нужен быстрый rollback приложения, но rollback схемы БД часто сложнее.

99. Практическая задача: как предотвратить двойную оплату?

Ответ: Использовать idempotency key, уникальное ограничение, транзакцию и state machine платежа. Внешний провайдер вызывается с собственным idempotency key, webhook обрабатывается повторяемо. Нельзя полагаться только на проверку «платёж не существует» без уникального ограничения — параллельные запросы пройдут одновременно.

```
1 <?php
2 $payment = $payments->findByKey($idempotencyKey);
3 if ($payment !== null) {
4     return $payment;
5 }
6
7 // UNIQUE(idempotency_key) защищает от гонки
8 return $payments->create($idempotencyKey, $amount);
```

100. Как рассказать о сложной задаче на собеседовании?

Ответ: Использовать структуру: контекст и масштаб, конкретная проблема, диагностика, принятое решение, компромиссы и измеримый результат. Отделять личный вклад от работы команды.

Сильный ответ содержит метрики и объясняет, почему выбрана именно эта архитектура, а не просто перечисляет технологии.

Что повторить перед собеседованием

- Система типов PHP 8.x: strict_types, union types, nullable types, enums, attributes и readonly.
- ООП и SOLID на практических примерах, а не только определения.
- HTTP-семантика, коды ответа, идиоматичность, JWT/OAuth2 и основные уязвимости.
- SQL, индексы, планы запросов, транзакции, уровни изоляции, N+1 и блокировки.
- Redis, стратегии кэширования, очереди, retries, DLQ, outbox и идиоматичные consumers.
- Жизненный цикл одного знакомого фреймворка: Laravel, Symfony или Yii2.
- Unit/integration tests, mocks/fakes, статический анализ и подход к code review.
- Архитектурные компромиссы: модульный монолит, микросервисы, eventual consistency и Saga.
- Практический рассказ о сложной задаче с метриками до и после изменения.

Мини-чеклист ответа

Хороший ответ Middle-разработчика обычно состоит из четырёх частей: определение, практический пример, возможная проблема и критерий выбора решения. Не обязательно рассказывать всё, что вы знаете: сначала дайте точный ответ, а затем углубляйтесь по уточняющим вопросам.